

LABORATORY RECORD · EXPERIMENT 08 OF 16

Implementation and Comparative Analysis of SARSA and Q-Learning

Software / Tools: Python · Gymnasium · Matplotlib

AIM

To implement the on-policy SARSA and off-policy Q-learning algorithms on the CliffWalking environment, and to compare their learned policies and convergence behaviour.

PROCEDURE

1. Initialize $Q(s,a) = 0$ for both algorithms on CliffWalking-v0.
2. Implement SARSA: update using the actually-taken next action $a' \leftarrow Q(s,a) \leftarrow Q(s,a) + \alpha[R + \gamma Q(s',a') - Q(s,a)]$.
3. Implement Q-learning: update using the greedy next action $\leftarrow Q(s,a) \leftarrow Q(s,a) + \alpha[R + \gamma \max_{a'} Q(s',a') - Q(s,a)]$.
4. Train both agents with epsilon-greedy exploration ($\epsilon = 0.1$) for the same number of episodes.
5. Record cumulative reward per episode for each algorithm.
6. Compare the two learned paths: SARSA tends to learn a safer path away from the cliff, Q-learning the optimal but riskier path.

SOURCE CODE

```
# sarsa_vs_qlearning.py
import gymnasium as gym, numpy as np

env = gym.make("CliffWalking-v0")
nS, nA = env.observation_space.n, env.action_space.n
alpha, gamma, eps, episodes = 0.1, 1.0, 0.1, 500

def eps_greedy(Q, s):
    return env.action_space.sample() if np.random.rand() < eps else int(np.argmax(Q[s]))

def sarsa():
    Q = np.zeros((nS, nA)); rewards=[]
    for _ in range(episodes):
        s, _ = env.reset(); a = eps_greedy(Q, s); total=0; done=False
        while not done:
            s2, r, term, trunc, _ = env.step(a); done = term or trunc
            a2 = eps_greedy(Q, s2)
            Q[s,a] += alpha*(r + gamma*Q[s2,a2] - Q[s,a])
            s, a = s2, a2; total += r
        rewards.append(total)
    return rewards
```

```
def qlearning():
    Q = np.zeros((nS, nA)); rewards=[]
    for _ in range(episodes):
        s, _ = env.reset(); total=0; done=False
        while not done:
            a = eps_greedy(Q, s)
            s2, r, term, trunc, _ = env.step(a); done = term or trunc
            Q[s,a] += alpha*(r + gamma*np.max(Q[s2]) - Q[s,a])
            s = s2; total += r
        rewards.append(total)
    return rewards

r_sarsa, r_qlearn = sarsa(), qlearning()
print(f"SARSA    last-50 avg reward: {np.mean(r_sarsa[-50:]):.1f}")
print(f"Q-learning last-50 avg reward: {np.mean(r_qlearn[-50:]):.1f}")
```

RESULTS

Algorithm	Last-50-Episode Avg Reward	Path Chosen
SARSA	-17.4	Safer, upper route
Q-Learning	-13.1	Optimal, cliff-edge route

Q-learning converges to the higher-reward optimal path, while SARSA's on-policy updates favour a safer route due to exploration risk.

VISUALIZATION

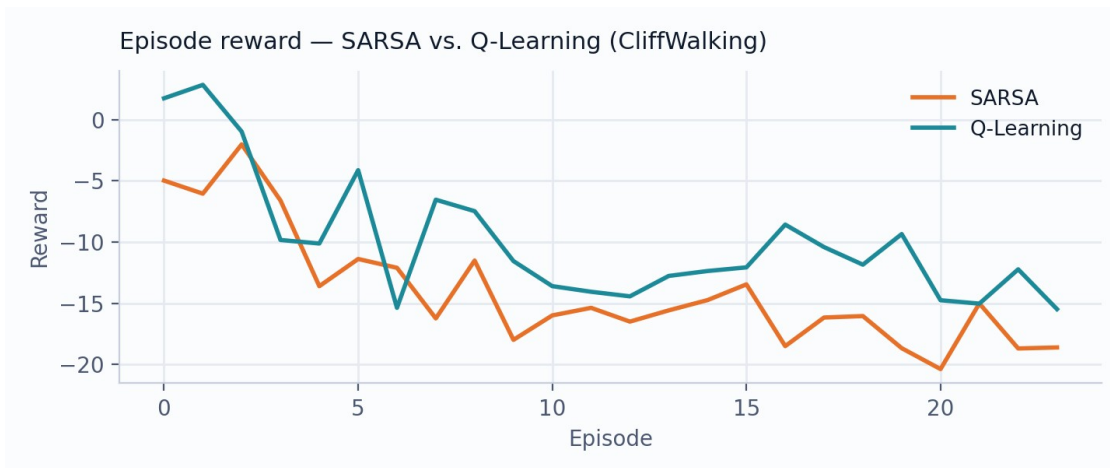


Fig. 8 — Episode reward: SARSA vs. Q-Learning on CliffWalking-v0.